```
#datasets used: Titanic, storms
#GATHER Commannd
View(Titanic)
help("Titanic")
Titanic_N=data.frame(Titanic)

install.packages("tidyr")
library(tidyr)
install.packages("dplyr")
library(dplyr)
#classwise count (row repetition to column heading)
#spread: No. of columns will increase
Data1=spread(Titanic_N, Class, Freq)
Data2=spread(Titanic_N, Class, Age)
Data4=spread(Titanic_N, Class, Sex)# Error: not a unique combination of keys observe 1,5 etc.

#classwise count (club rows 4 to 7) i. e column to row
#gather: No. of columns will decrease
Data3=gather(Data1, "Class", "Freq", 4:7)

# unite and separate
#Split and merge columns with unite() and separate()

View(storms)
storms=data.frame(storms)
storms_club=unite(storms, "date", year, month, day, sep = "-")
storms_seperate=separate(storms_club, date, c("year", "month", "day"), sep = "-")

# Practice commands on "marketing_campaign" data set
# Installation of Required Packages
install.packages("tidyr")
library(tidyr)
#Import "marketing_campaign" data set
View(marketing_campaign)
str(marketing_campaign)
summary(marketing_campaign)

# Collapsing    multiple        columns into    two     columns

## Example 1
Y = gather(marketing_campaign, "Cmp", "n", 21:25)
View(Y)
# Observe the last two columns of new data set Y. Also observe the change in
# total number of columns

## Example 2
X = gather(marketing_campaign, "PurchasesType", "Quantity", 16:19)
View(X)
# Observe the last two columns of new data set Y. Also observe the change in
# total number of columns


# Generates     multiple        columns from    two     columns

## Example 1
Y1 = spread(marketing_campaign, Marital_Status, Income)
View(Y1)
# Observe the column numbers 28 - 35 of new data set Y1. Also observe the change
# in total number of columns

## Example 2
Y2 = spread(marketing_campaign, Education, Income)
View(Y2)
# Observe the column numbers 28 - 32 of new data set Y1. Also observe the change
# in total number of columns

# splitting a   column  by      a       character       string  separator
Y3 = separate(marketing_campaign, Dt_Customer, c("day", "month", "year"),
              sep = "-")
```

```
View(Y3)
# Observe the column numbers 8 - 10 of new data set Y2. Also observe the change
# in total number of columns

# Uniting multiple      columns into    a       single  column
Y4 = unite(Y2, "Dt_Customer", year, month, day, sep = "-")
View(Y4)
# Observe the column number 8 of new data set Y3. Also observe the change
# in total number of columns


#dplyr
install.packages("dplyr")
library(dplyr)
data(storms)
View(storms)
str(storms)
summary(storms)

#select: extraction by column
s1=select(storms, status, wind)
s2=select(storms,-status,-wind)
s3=select(storms,name:day)
s4=select(storms,2:4)
s5=select(storms,1,8,10,11)
s6=select(storms, contains("force"))

#filter: extraction by row
f1=head(storms,10)#first 10 rows, default shows 6 rows
tail(storms)#bottom 6 rows
slice(f1,2:6) #rows from 2 to 6
f2=filter(f1,wind>=30)
f3=filter(f1,wind>=30,status=="tropical storm")
f4=filter(f1, wind      >=      30, status      %in%    c("tropical storm"))

#for calculations from existing columns using mutate
m1=mutate(f1, ratio = pressure / wind)
m2=mutate(f1,minute=hour/60)

#arrange in ascending or descending
B=arrange(storms,pressure)
C=arrange(storms,desc(pressure))

#summarise: to change the unit (big data to small values)
summarise(f1,standard_Dev=sd(lat), avg=mean(lat),count=n())

#practice purpose
View(mtcars)
mtcars=data.frame(mtcars)
str(mtcars)
summary(mtcars)
mean(mpg)#Calulates average of mpg column and O/P in console window
Avg_mpg=summarise(mtcars,avg=mean(mpg))#creates new datasheet with value
auto_yn=group_by(mtcars,am)#creates groups as per "am" column
auto_yn1=summarise(auto_yn,avg=mean(mpg),Number=n())

#pipe operator
mtcars$mpg %>% mean()#20.09062 vector
mtcars %>% select(mpg) %>% mean()# data frame not vector
             #warning:argument is not numeric or logical: returning NA
mtcars %>% select(mpg) %>% summarise(avg=mean(mpg),number=n())

#same steps done for line 122,123 can be done using pipe operator as
mtcars %>% group_by(am) %>% summarise(avg=mean(mpg),number=n())
mtcars %>% group_by(cyl) %>% summarise(avg=mean(mpg))
New= group_by(mtcars,am) %>% summarise(avg=mean(mpg))
N=storms %>% select(pressure,wind) %>% filter(wind>=30)
storms %>% summarise(mean = mean(hour), sum = sum(hour), count = n())
```

```r
#missing values
A=c(1,3,5,NA,"AB")
min(A)#Error
mean(A)#Error
mean(A,na.rm=TRUE)#ans :3
anyNA(A)#TRUE
which(is.na(A))#position of NA
A[is.na(A)]=0 #replace NA by 0
# Filter out missing values
#Run A again
new_A <- A[!is.na(A)] # new data frame with no missing values
print(new_A)
# Output: 1 3 5

#practice
View(airquality)
head(airquality)        # First few rows
summary(airquality)     # Summary showing NA counts
anyNA(airquality)       # Checks if any NA exists
is.na(airquality)       # TRUE/FALSE for each cell
sum(is.na(airquality))  # Total number of missing values
colSums(is.na(airquality))   # Count of NAs per column
sapply(airquality, function(x) sum(is.na(x)))  # Alternative
which(is.na(airquality))   #index positions (in vector form)
na.omit(airquality)        # Remove rows with any NA
#Replace NA in Ozone with the column mean (ignoring NAs)
airquality$Ozone[is.na(airquality$Ozone)] = mean(airquality$Ozone, na.rm = TRUE)
airquality$Ozone    # NA's are replaced
View(airquality)

# For Particase
data("airquality")
airquality$Ozone[is.na(airquality$Ozone)] = 0
airquality %>% filter(Ozone==0)# shows only rows with Ozone=0

#trimws
a=data.frame(Names=c("P","Q","R","s"),
             Marks=c(30,"AB",10,"?"),
             Gender=c("Male", " ","","Male"))
is.na(a)
a$Gender==""  #shows True for vacant
a$Gender==" "  #shows True for space
trimws (a$Gender) == ""  #shows True for vacant and space both
summary(a)
a$Marks=as.numeric(a$Marks)
sum(a$Marks,na.rm=TRUE)

# Make a copy of the dataset
aq= airquality
# Convert 'Ozone' to character so we can insert blanks (NA can stay in numeric)
aq$Ozone <- as.character(aq$Ozone)
anyNA(aq)                #check for NA
is.na(aq$Ozone)          # Only original NA values will be TRUE
#create a new data with empty string and space
# Introduce empty strings and spaces in place of some values
aq$Ozone[1] <- ""     # Assign first element as empty string
aq$Ozone[2] <- " "    # Assign second element as space
aq$Ozone == ""           # TRUE for exact empty strings only
trimws(aq$Ozone) == ""   # TRUE for empty strings and spaces
aq$Ozone[trimws(aq$Ozone) == ""] <- NA
is.na(aq$Ozone)

#datatype change
class(airquality$Ozone)     # Data type of one column
str(airquality)             # Structure of whole dataset
typeof(airquality$Ozone)    # Low-level storage type
airquality$Ozone <- as.character(airquality$Ozone)
class(airquality$Ozone)
airquality$Ozone <- as.integer(airquality$Ozone)
```

```
airquality$Month <- as.factor(airquality$Month)
#With custom labels:
airquality$Month <- factor(airquality$Month,
                           levels = 5:9,
                           labels = c("May", "Jun", "Jul", "Aug", "Sep"))
typeof(airquality$Month)
class(airquality$Month)
View(airquality)



#Additional commands
#pointwise min of columns
a=data.frame(pmin(storms$month, storms$hour))
n_distinct(f1$hour) #The number of distinct values in a vector

#data join
Div_A=data.frame(PRN=c(120, 125, 126), Name=c("Ramesh", "Radha", "Aasha"), SGPA=c(6.7, 7.8, 8))
Div_B=data.frame(PRN=c(127, 128, 129), Name=c("Aaroh", "Rima", "mala"), SGPA=c(7, 7.5, 6))
Result=rbind(Div_A, Div_B)
Div_A_Course=data.frame(LAUC=c(23, 22, 12), Phy=c(12, 17, 20))
Div_A_Result=cbind(Div_A, Div_A_Course)
FA=data.frame(PRN=c(123, 124, 125, 126, 127),
              Name=c("Sudha","Prathmesh", "Anita", "Radha", "Mahesh"),FA1_marks=c(12, 23, 21,
22, 10), FA2_marks=c(15, 20, 20, 15, 16))

SA=data.frame(PRN=c(123, 124, 125,  127,128),
              SA_marks=c(16, 14, 10,  15, 18), Result=c("P","P","F","P", "P" ))
# semi_join(x, y, by = "key")   # Keep only rows in x that have a match in y; return columns
from x only

Final_result_S=semi_join(FA, SA, join_by(PRN))
# inner_join(x, y, by = "key")  # Keep only rows that match in both x and y; return columns
from both
Final_result_i=inner_join(FA, SA)
# left_join(x, y, by = "key")   # Keep all rows from x, match with y when possible; return
columns from both
Final_result_l=left_join(FA, SA)
# right_join(x, y, by = "key")  # Keep all rows from y, match with x when possible; return
columns from both
Final_result_r=right_join(FA, SA)
# anti_join(x, y, by = "key")   # Keep only rows in x that have NO match in y; return columns
from x only
Final_result_a=anti_join(FA, SA)

library(dplyr)

students <- data.frame(
  id = c(1, 2, 3, 4),
  name = c("Anita", "Rahul", "Sneha", "Amit"))

marks <- data.frame(
  id = c(2, 3, 4,5),
  marks = c(88, 92, 75,85))

students_i =inner_join(students, marks, by = "id")#all columns but common rows
students_s =semi_join(students, marks, by = "id")
students_l=left_join(students, marks, by = "id")
students_r=right_join(students, marks, by = "id")
students_a=anti_join(students, marks, by = "id")
students_m=anti_join(marks, students, by = "id")

#from test 1
data("PlantGrowth")
data("CO2")
View(CO2)
unique(CO2$Plant)      #op as different levels
n_distinct(CO2$Plant)  #count different types in column OP:12
```

```
str(CO2$Plant)
filter(CO2,Treatment=="chilled") %>% group_by(Plant) %>% summarise(Avg=mean(uptake))
```